

Arm[®] GPU Bug Bounty Virtual Platform How-To Guide

Public

Copyright © 2025 Arm Limited (or its affiliates). All rights reserved.

Document Version: 20250623-1.0

- [Setting up a Virtual Platform with Arm[®] Mali[™] Drivers](#)
 - [General Recommendations for Virtual Platforms](#)
 - [Integrating Mali into the Linux Kernel](#)
 - [Testing with some example code](#)
 - [Troubleshooting](#)
- [Proprietary notice](#)

This document is Non-Confidential.

Copyright © 2025 Arm Limited or its affiliates. All rights reserved.

This document is protected by copyright and other intellectual property rights. Arm only permits use of this document if you have reviewed and accepted [Arm's Proprietary notice](#) found at the end of this document.

These materials and information are provided as part of Arm's Bug Bounty program to assist security researchers in their vulnerability research related to this program. These documents do not constitute official Arm documentation, and no warranties or guarantees are provided regarding their accuracy, completeness, or suitability for any purpose outside of the Bug Bounty context. Arm reserves the right to modify or withdraw any materials at its discretion without prior notice.

Setting up a Virtual Platform with Arm[®] Mali[™] Drivers

If you don't have a physical device with a Mali GPU present, you can setup a Virtual Platform with a simple Mali model known as the "dummy model" (the "No Mali" configuration) using the Mali 5th Gen drivers.

The Mali GPU Kernel Driver (mali_kbase.ko) must be built and loaded onto this Virtual Platform by following the instructions below.

The "dummy model" configuration does not attempt to request or execute the GPU Firmware, therefore testing against a Virtual Platform is limited to the GPU driver ("GPU Software") and you will be unable to test against the Firmware or Hardware.

Note: If this is your first time setting up a Linux Kernel, then please visit <https://kernelnewbies.org/KernelBuild> for more information.

Note: If you are new to Virtual Platforms, please visit: Running Linux in a VM - Kernel Newbies Community: <https://kernelnewbies.org/OutreachyfirstpatchAlt>

The following variable will be used to refer to the directory containing your Linux Kernel source:

```
KDIR
```

and on successfully setting up, will contain the path:

```
$KDIR/arch/arm64
```

There are two paths to running Mali drivers on a Linux Virtual Platform

1. "Device Tree" configurations
2. "Simulated Platform Device" configurations, as supported on the Versatile Express platform

The "Device Tree" configuration is generally recommended for small-form-factor physical devices such as embedded devices, smartphones, tablets, Single-board-Computers, and so on. It can be used on Arm-based Virtual Platforms by providing a custom-built dtb file. Steps for this are provided in [Configuring Device Tree on Arm Virtual Platforms](#).

The "Simulated Platform Device" configuration can be used for Workstation Virtual Platforms (which include x86 systems), where there is no "Device Tree" present. This requires more steps to be taken than "Device Tree" configurations, including modifying the Mali driver. For those using an x86 system, the steps to apply the required Mali driver patches are described in [Patching the Mali Driver Source to enable x86 builds](#)

General Recommendations for Virtual Platforms

The below instructions have been tested on Ubuntu Linux 24.04.2 LTS running both Arm aarch64 amd x86_64 with:

- 16GB RAM
- 256GB disk space
- 4 or more vCPUs

Of course, the more vCPUs are allocated to the guest the faster the compile time of the Linux Kernel will be. On an Apple M3 Max with 12 vCPUs assigned to the Virtual Platform the Kernel will compile in minutes, whereas on a 4-core i7-4770 you can expect Kernel compiles to take many hours.

Excluding compiling the kernel and other tools, installing the Mali drivers is under an hour's work.

Integrating Mali into the Linux Kernel

Downloading and unpacking the Mali Driver Source

For Virtual Platforms using the 'dummy' Mali GPU, you should download the Mali 5th Gen GPU driver ('Kbase') from:

- <https://developer.arm.com/downloads/-/mali-drivers/5th-gen-gpu-architecture-kernel>

Use the latest version available.

Untar the Kbase source into a new directory.

The following variable will be used to refer to that directory:

```
MALI_DIR
```

and on successfully unpacking, will contain the path:

```
$MALI_DIR/driver/product/kernel
```

Integrating the Mali Driver Source into your Linux Kernel

Copy the Kbase Source into the Kernel tree for an in-tree build as follows:

```
# See above sections for settings of MALI_DIR and KDIR
cp -a "$MALI_DIR"/driver/product/kernel/* "$KDIR/"
```

One of the kernel's **Makefile** files and **Kconfig** files also need modifying to correctly integrate Mali into the kernel build:

```
cd $KDIR
echo 'obj-$(CONFIG_MALI_MIDGARD) += arm/' | tee -a drivers/gpu/Makefile
sed -i '$i source "drivers/gpu/arm/Kconfig"' drivers/video/Kconfig
```

[Arm/"Device Tree" only] Configuring the Linux Kernel for Mali

Set the following in your Linux Kernel, e.g. run `make menuconfig` and choose the options found under Device Drivers → Graphics Support → ARM GPU Configuration

```
CONFIG_MALI_MIDGARD=m
CONFIG_MALI_CSF_SUPPORT=y
CONFIG_MALI_EXPERT=y

# This is under:
# -> "Mali HW backend"
#   -> "Enable build of Mali kernel driver for No Mali"
CONFIG_MALI_NO_MALI=y

# CONFIG_MALI_REAL_HW is not set

# No Mali Model for G725/G925 GPU
CONFIG_MALI_NO_MALI_DEFAULT_GPU="tKRx"

CONFIG_MALI_PLATFORM_NAME="devicetree"

# Note that CONFIG_LARGE_PAGE_SUPPORT=y will be selected by
# default
```

This will:

- Enable the Kbase driver as a Module
- Build the driver for "CSF" GPUs
- Allow for further 'Expert' configs to be used
- Use the Mali "dummy model" ("No Mali")
- Default the "dummy model" to be a Mali-G925
- Use the Device Tree integration to allow mali_kbase to locate the dummy GPU
 - **Note:** you must follow the steps in [Configuring Device Tree on Arm Virtual Platforms](#) for this to work

[Arm/"Device Tree" Only] Configuring Device Tree on Arm Virtual Platforms

Configuring the Virtual Platform for an Arm CPU requires no modifications to the Mali Kernel driver, and uses Device Tree to probe for devices. x86 Virtual Platforms tend not use Device Tree and instead you must follow the steps for the ["Simulated Platform Device" configuration](#) and [Patching the Mali Driver Source to enable x86 builds](#)) below.

Mali's Device Tree node can be integrated into the Virtual Platform's Device Tree Blob ('DTB') as follows:

1. Install `dtc` (device-tree-compiler) either in your Linux Virtual Platform:

```
apt install device-tree-compiler
```

or on your host, such as the following for MacOS:

```
brew install dtc
```

2. Shutdown your Virtual Platform and dump your existing default dtb, e.g. re-run your existing Virtual Platform with an additional parameter such as:

```
-dumpdtb=original.dtb
```

3. Locate the dtb
 - if the GUI to your Virtual Platform isn't clear on where these are stored, try locating it with:

```
find ~ -type f -name '*.dtb' 2>/dev/null
```

4. Decompile the dtb into a dts by running:

```
dtc -I dtb -O dts original.dtb -o original.dts
```

- if `dtc` is not already on your `PATH`, then copy the dtb to wherever you have `dtc` installed

5. Edit the dts and insert the following lines before the final `};`:

```
gpu: gpu@6e000000 {
    compatible = "arm,mali-midgard";
    reg = <0x0 0x6e000000 0x0 0x200000>;
    interrupts = <0 168 4>, <0 168 4>, <0 168 4>;
    interrupt-names = "JOB", "MMU", "GPU";
};
```

6. Save this as a new file (e.g., `dummy_mali.dts`)

7. Compile a replacement dtb by running:

```
dtc -I dts -O dtb dummy_mali.dts -o dummy_mali.dtb
```

- If `dtc` was not already on your `PATH`, place the resulting `dummy_mali.dtb` file in the same directory as the original dtb file.

8. Ensure ACPI is disabled and instruct your Virtual Platform to use the newly compiled dtb. For example:

- Add `acpi=off` to the `machine` parameter
- Add a new argument of `-dtb` and a new value of `dummy_mali.dtb`

9. Save these settings and start the Virtual Platform

[x86/"Simulated Platform Device" only] Configuring the Linux Kernel for Mali

Set the following in your Linux Kernel, e.g. run `make menuconfig` and choose the options found under Device Drivers → Graphics Support → ARM GPU Configuration

```
CONFIG_MALI_MIDGARD=m
CONFIG_MALI_CSF_SUPPORT=y
CONFIG_MALI_EXPERT=y

# This is under:
# -> "Mali HW backend"
#   -> "Enable build of Mali kernel driver for No Mali"
CONFIG_MALI_NO_MALI=y

# CONFIG_MALI_REAL_HW is not set

# No Mali Model for G725/G925 GPU
CONFIG_MALI_NO_MALI_DEFAULT_GPU="tKRx"

CONFIG_MALI_PLATFORM_NAME="vexpress"

# Note that CONFIG_LARGE_PAGE_SUPPORT=y will be selected by default
```

This will:

- Enable the Kbase driver as a Module
- Build the driver for "CSF" GPUs
- Allow for further 'Expert' configs to be used
- Use the Mali "dummy model" ("No Mali")
- Default the "dummy model" to be a Mali-G925
- Use the Versatile Express platform file, which includes a "Simulated Platform Device" in non-Device Tree builds

[x86/"Simulated Platform Device" only] Patching the Mali Driver Source to enable x86 builds

Note: No patches are required when configuring the Virtual Platform for an Arm CPU: [Configuring Device Tree on Arm Virtual Platforms](#)

A number of patches are provided in the following attachment:

`patches_for_virtual_device.zip`

These enable use of the Mali Driver on non-Arm CPU Virtual Devices.

The patches apply cleanly to the Mali 5th Gen GPU Kernel Driver source, version r54p0. They may need to be ported to work on other driver versions.

Extract the zip file into `$KDIR`, and apply each one. For example:

```
cd "$KDIR"
for patch_file in *.patch; do
    echo "$patch_file"
```

```
patch -p3 -i "$patch_file"
done
```

Refer to the sections below for details of each required modification.

Note: an additional patch is provided that applies both to Arm and x86 builds. Refer to the section on [Troubleshooting](#) for more details.

[x86/"Simulated Platform Device" only] Correct the KERNEL_VERSION guards on of_property*_flag functions

Current versions of Kbase are not designed for use on v4.1+ kernels when devicetree is disabled (`CONFIG_OF=n`), e.g. in the case of a non-Arm CPU.

The Kernel version guard needs to be corrected from "`>= v4.15`" to "`< v4.1`"

Without this modification, the following problem may be seen:

```
In file included from ./drivers/gpu/arm/midgard/mali_kbase.h:31,
                  from drivers/gpu/arm/midgard/mali_kbase_cache_policy.c:27:
./include/linux/version_compat_defs.h:568:19: error: conflicting types for
'of_property_check_flag'
  568 | static inline int of_property_check_flag(const struct property *p,
      |                                     ^~~~~~
In file included from ./include/linux/version_compat_defs.h:33,
                  from ./drivers/gpu/arm/midgard/mali_kbase.h:31,
                  from drivers/gpu/arm/midgard/mali_kbase_cache_policy.c:27:
./include/linux/of.h:971:19: note: previous definition of
'of_property_check_flag' was here
  971 | static inline int of_property_check_flag(struct property *p,
      |                                     ^~~~~~
compilation terminated.
```

The following patch will modify the driver to handle this:

- `0001-mali-fix-build-error-for-CONFIG_OF=n-for-4.1-kernels.patch`

[x86/"Simulated Platform Device" only] Remove asm/arch_timer.h and provide replacement arch_timer functions

The `arch_timer.h` interface for the Arm system timer is unavailable on non-Arm CPUs.

For the purposes of this Program, it is acceptable to remove use of `asm/arch_timer.h`, and replace `arch_timer_get_cntfrq()` with a non-zero constant.

Without this modification, the following problems may be seen:

```
In file included from drivers/gpu/arm/midgard/mali_kbase_core_linux.c:98:
./include/linux/mali_hw_access.h:25:10: fatal error: asm/arch_timer.h: No
such file or directory
  25 | #include <asm/arch_timer.h>
      |          ^~~~~~
compilation terminated.
```

and/or:

```

In file included from drivers/gpu/arm/midgard/backend/gpu/mali_kbase_time.c:
34:
drivers/gpu/arm/midgard/backend/gpu/mali_kbase_time.c: In function
'kbase_arch_timer_get_cntfrq':
./include/linux/mali_hw_access.h:39:38: error: implicit declaration of
function 'arch_timer_get_cntfrq'; did you mean 'mali_arch_timer_get_cntfrq'?
[-Werror=implicit-function-declaration]
   39 | #define mali_arch_timer_get_cntfrq() arch_timer_get_cntfrq()
      |                                     ^~~~~~
drivers/gpu/arm/midgard/backend/gpu/mali_kbase_time.c:332:13: note: in
expansion of macro 'mali_arch_timer_get_cntfrq'
   332 |     u64 freq = mali_arch_timer_get_cntfrq();
      |             ^~~~~~

```

The following patches will modify the driver to handle this:

- [0002-Fix-x86-build-error-for-missing-asm-arch_timer.h.patch](#)
- [0003-Workaround-arch_timer-funcs-undefined-for-NO_MALI.patch](#)

[x86/"Simulated Platform Device" only] Provide a definition of dmb()

The `dmb()` macro to issue a data memory barrier is unavailable to non-Arm CPUs.

For the purposes of this Program, it is acceptable to replace these with an `mb()` call to issue a general memory barrier.

Without this modification, the following problem may be seen:

```

drivers/gpu/arm/midgard/csf/mali_kbase_csf.c: In function
'kbase_csf_ring_csg_slots_doorbell':
drivers/gpu/arm/midgard/csf/mali_kbase_csf.c:873:2: error: implicit
declaration of function 'dmb'; did you mean 'rmb'? [-Werror=implicit-
function-declaration]
   873 |     dmb(osh);
      |     ^~~
      |     rmb
drivers/gpu/arm/midgard/csf/mali_kbase_csf.c:873:6: error: 'osh' undeclared
(first use in this function)
   873 |     dmb(osh);
      |         ^~~

```

The following patches will modify the driver to handle this:

- [0004-Workaround-no-definition-of-dmb-in-non-Arm-platforms.patch](#)

[x86/"Simulated Platform Device" only] Remove unused functions

There are a number of functions that are only ever used when devicetree is enabled (`CONFIG_OF=y`), e.g. in the case of a Arm CPU.

If a newer kernel is used that enables `-Werror` and `-Wunused-function`, then the build may fail on attempting to compile files with these unused functions.

The driver should be modified to remove the unused functions, or guard them with C pre-processor macros.

Without this modification, the following problems may be seen:

```
CC [M] drivers/gpu/arm/midgard/device/mali_kbase_device.o
drivers/gpu/arm/midgard/device/mali_kbase_device.c:130:12: warning:
'pcm_prioritized_process_cb' defined but not used [-Wunused-function]
  130 | static int pcm_prioritized_process_cb(struct notifier_block *nb,
      |          ^~~~~~
      |          ^~~~~~
CC [M] drivers/gpu/arm/midgard/backend/gpu/mali_kbase_devfreq.o
drivers/gpu/arm/midgard/backend/gpu/mali_kbase_devfreq.c:350:13: warning:
'kbasep_devfreq_read_suspend_clock' defined but not used [-Wunused-function]
  350 | static void kbasep_devfreq_read_suspend_clock(struct kbase_device
      |          ^~~~~~
      |          ^~~~~~
*kbdev, struct device_node *node)
```

The following patches will modify the driver to handle this:

- [0005-Fix-unused-function-warnings.patch](#)

Build and install the Mali GPU Kernel Driver

The Mali GPU Kernel Driver is built in the usual way for modules inside the Linux Kernel, for example:

```
ARCH=... CROSS_COMPILE=... make modules
```

mali_kbase.ko is then installed onto the device in the usual way.

A successful `insmod` of the mali kernel driver build for the "dummy model" looks like this on dmesg:

```
mali mali.0: Kernel DDK version r54p0-00eac0
mali mali.0: Using Dummy Model
mali mali.0: GPU metrics tracepoint support enabled
mali mali.0: Register LUT 000c0000 initialized for GPU arch 0x000d0801
mali mali.0: GPU identified as 0x0 arch 13.8.1 r0p0 status 0
mali mali.0: No OPPs found in device tree! Scaling timeouts using 100000 kHz
mali mali.0: Large page allocation set to true after hardware feature check
Dummy model register access: Writing unsupported register 0x88 value 0x0
Dummy model register access: Writing unsupported register 0x8c value 0x0
Dummy model register access: Writing unsupported register 0x88 value
0xf8f58abb
Dummy model register access: Writing unsupported register 0x8c value 0x2
mali mali.0: Clock not available for devfreq
mali mali.0: Continuing without devfreq
mali mali.0: Probed as mali0
```

Testing with some example code

Arm's libGPUCounters can be used to test basic operation of Kbase.

Download and compile libGPUCounters example

Download libGPUCounters:

```
git clone https://github.com/ARM-software/libGPUCounters.git
```

Download cmake - must be least version 3.13.5:

```
sudo apt-get install cmake  
cmake -version
```

Setup to build the example, as per the [README.md](#):

```
cd libGPUCounters  
cmake -DHWCPipe_BUILD_EXAMPLES=ON -B build .
```

expected output:

```
-- The C compiler identification is GNU 9.4.0  
-- The CXX compiler identification is GNU 9.4.0  
-- Check for working C compiler: /usr/bin/cc  
-- Check for working C compiler: /usr/bin/cc -- works  
-- Detecting C compiler ABI info  
-- Detecting C compiler ABI info - done  
-- Detecting C compile features  
-- Detecting C compile features - done  
-- Check for working CXX compiler: /usr/bin/c++  
-- Check for working CXX compiler: /usr/bin/c++ -- works  
-- Detecting CXX compiler ABI info  
-- Detecting CXX compiler ABI info - done  
-- Detecting CXX compile features  
-- Detecting CXX compile features - done  
-- Configuring done  
-- Generating done  
-- Build files have been written to: /work/libGPUCounters/build
```

Build the example:

```
cd build  
make -j4
```

Expected output:

```
Scanning dependencies of target device  
[ 5%] Building CXX object backend/device/CMakeFiles/device.dir/src/device/  
handle.cpp.o  
[ 10%] Building CXX object backend/device/CMakeFiles/device.dir/src/device/  
hwcnt/backend_type.cpp.o  
[ 15%] Building CXX object backend/device/CMakeFiles/device.dir/src/device/
```

```

hwcnt/reader.cpp.o
[ 21%] Building CXX object backend/device/CMakeFiles/device.dir/src/device/
hwcnt/sampler/detail/backend.cpp.o
[ 26%] Building CXX object backend/device/CMakeFiles/device.dir/src/device/
hwcnt/sampler/kinstr_prfcnt/block_index_remap.cpp.o
[ 31%] Building CXX object backend/device/CMakeFiles/device.dir/src/device/
hwcnt/sampler/kinstr_prfcnt/enum_info_parser.cpp.o
[ 36%] Building CXX object backend/device/CMakeFiles/device.dir/src/device/
hwcnt/sampler/kinstr_prfcnt/metadata_parser.cpp.o
[ 42%] Building CXX object backend/device/CMakeFiles/device.dir/src/device/
instance.cpp.o
[ 47%] Building CXX object backend/device/CMakeFiles/device.dir/src/device/
num_exec_engines.cpp.o
[ 52%] Building CXX object backend/device/CMakeFiles/device.dir/src/device/
product_id.cpp.o
[ 57%] Linking CXX static library libdevice.a
[ 57%] Built target device
Scanning dependencies of target hwcpipeline
[ 63%] Building CXX object hwcpipeline/CMakeFiles/hwcpipeline.dir/src/error.cpp.o
[ 68%] Building CXX object hwcpipeline/CMakeFiles/hwcpipeline.dir/src/hwcpipeline/detail/
counter_database.cpp.o
[ 73%] Building CXX object hwcpipeline/CMakeFiles/hwcpipeline.dir/src/hwcpipeline/
all_gpu_counters.cpp.o
[ 78%] Building CXX object hwcpipeline/CMakeFiles/hwcpipeline.dir/src/hwcpipeline/
counter_metadata.cpp.o
[ 84%] Building CXX object hwcpipeline/CMakeFiles/hwcpipeline.dir/src/hwcpipeline/
derived_functions.cpp.o
[ 89%] Linking CXX static library libhwcpipeline.a
[ 89%] Built target hwcpipeline
Scanning dependencies of target api-example
[ 94%] Building CXX object examples/CMakeFiles/api-example.dir/
api_example.cpp.o
[100%] Linking CXX executable api-example
[100%] Built target api-example

```

Run the libGPUCounters example

From the build directory:

```
examples/api-example
```

Example output:

```

-----
GPU Device 0:
-----
    Product Family:  Arm 5th Gen
    Number of Cores: 13
    Bus Width:       64
-----
GPU 0 Supported counters:
-----

```

This output will indicate that the Mali Driver Kernel Module has been built and installed into the system properly.

Note the counter values are expected to be 0 for the "dummy model".

If there was a problem with accessing the /dev/mali0 device node, then the following will be reported:

```
Mali GPU device 0 is missing
```

Troubleshooting

Build error: `make clean` errors when Mali has been integrated into the kernel

Possible error output:

```
scripts/Makefile.clean:15: drivers/gpu/arm/midgard/./arbitration/Makefile:
No such file or directory
make[5]: *** No rule to make target 'drivers/gpu/arm/midgard/./arbitration/
Makefile'. Stop.
make[4]: *** [scripts/Makefile.clean:67: drivers/gpu/arm/midgard/./
arbitration] Error 2
make[3]: *** [scripts/Makefile.clean:67: drivers/gpu/arm/midgard] Error 2
make[2]: *** [scripts/Makefile.clean:67: drivers/gpu/arm] Error 2
make[1]: *** [scripts/Makefile.clean:67: drivers/gpu] Error 2
make: *** [Makefile:1785: _clean_drivers] Error 2
```

Whilst the Mali Kernel Driver can be built successfully using `make`, cleaning the kernel with `make clean` fails.

This is due to the Mali GPUs Arbitration Reference Code not being included in the setup. However in most cases the Arbitration code is not required, such as in testing systems where only a single Guest OS is using the GPU.

The following patch file fixes this:

- [0006-Fix-make-clean-when-no-arbitration-code-present.patch](#)

No dmesg output after `insmod` of `mali_kbase.ko` / `kbase` doesn't work on Arm Virtual Devices

Whilst `lsmod` may output the module being present:

```
$ sudo lsmod | grep mali
mali_kbase          2125824  0
```

...there may be no dmesg output on `insmod`, and `kbase` can't be used.

This can happen when the Virtual Device is for an Arm CPU/is configured for Device Tree, and there is no Mali Device Tree Node to allow probing for the Mali Driver.

The options to get this working are:

- Create a `.dts` file for Kbase, and include it in your dtb
 - Refer to [\\$MALI_DIR/driver/product/kernel/Documentation/devicetree/bindings/arm/mali-midgard.txt](#) for the format of the Device Tree Source for Kbase, and an example.
 - Refer to [Configuring Device Tree on Arm Virtual Platforms](#) for how to integrate the resulting Device Tree source.
- Or, force use of the "Simulated Platform Device", and disable any Device Tree usage in the driver
 - The "Simulated Platform Device" is compiled in for `CONFIG_MALI_PLATFORM_NAME="vexpress"`

Copyright © 2025 Arm Limited (or its affiliates). All rights reserved.

Public

- However, there are additional problems:
 - On Device Tree platforms, the "Simulated Platform Device" is not used, so you have to manually enable it.
 - In addition, the remaining Device Tree integration in the driver will fault with an Unhandled NULL pointer access.
- To resolve both of these issues, search for all uses of `CONFIG_OF` in the driver, and update them as follows:
 - `!IS_ENABLED(CONFIG_OF) → 1`
 - `IS_ENABLED(CONFIG_OF) → 0`
 - `#ifndef CONFIG_OF → #if 1`
 - `#ifdef CONFIG_OF → #if 0`
 - `#if !defined(CONFIG_OF) → #if 1`
 - `#if defined(CONFIG_OF) → #if 0`
 - `of_match_ptr(kbase_dt_ids) → NULL`
- This will force the driver to use non-Device Tree code paths on a Device Tree system
- There is no patch provided for this, as the locations to change can be different in every version of the Mali Driver.

Proprietary notice

This document is protected by copyright and other related rights and the practice or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). **No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.**

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of the document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any use, duplication or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership

relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. All rights reserved. Please follow Arm's trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.
110 Fulbourn Road, Cambridge, England CB1 9NJ.
(PRE-1121-V1.0)